

WORKED EXAMPLE

Choose when to stop and what to restore

Lecture 5 page 160 motivates early stopping with held-out validation performance.

Training loss guides parameter gradients. Validation loss guides the stated checkpoint and stopping rule. The test set is reserved for final evaluation, outside repeated run or model selection.

A checkpoint stores the actual model parameters at an epoch, not just its loss number. Stopping at one epoch does not mean its parameters are the best saved ones.

The lecture gives the concept; the precise patience rule below is chosen for this study example. Different declared rules can stop at different epochs.

WORKED EXAMPLE

State the rule before reading future losses

Lower validation loss is better; $\text{min_delta}=0$ and $\text{patience}=2$.

Improvement means a strict decrease below the best validation loss seen so far. Equality is not improvement. With $\text{min_delta}=0$, even a tiny strict decrease counts.

On improvement: replace the best loss, save that epoch checkpoint and reset the consecutive-failure counter to zero.

Otherwise: leave the best checkpoint unchanged and add one to the counter. Stop immediately after an epoch makes the counter reach 2.

Restore the saved best checkpoint for evaluation. Because every strict decrease counts, the checkpoint reference and the patience reference are the same best loss.

$\text{improve} \iff v_e < v_{best}; \quad \text{stop} \iff \text{bad count} \geq 2$

WORKED EXAMPLE

Read the supplied training and validation history

Each row is evaluated after completing that epoch.

epoch	training loss	validation loss
1	.65	.60
2	.50	.50
3	.42	.51
4	.35	.49
5	.30	.495
6	.25	.50

Training loss falls throughout. The rule will still stop when validation loss fails to improve for two consecutive epochs. Read rows in time order; do not choose actions using later values.

WORKED EXAMPLE

Epochs 1 and 2: save improvements

Compare each new validation value with the best seen so far.

Epoch 1 is the first observation: save its checkpoint and initialize the best validation loss.

best loss = 0.6, checkpoint = 1, counter = 0

epoch 2 : $0.5 < 0.6 \implies$ improves

best loss = 0.5, checkpoint = 2, counter = 0

Saving a new best checkpoint resets the counter; a failure does not erase the previously saved parameters.

WORKED EXAMPLE

Epochs 3 and 4: fail, then reset

Compare each new validation value with the best seen so far.

epoch 3 : $0.51 > 0.5 \implies$ no improvement

best loss = 0.5, checkpoint = 2, counter = 1

epoch 4 : $0.49 < 0.5 \implies$ improves

best loss = 0.49, checkpoint = 4, counter = 0

Saving a new best checkpoint resets the counter; a failure does not erase the previously saved parameters.

WORKED EXAMPLE

Epochs 5 and 6: patience is exhausted

Compare each new validation value with the best seen so far.

epoch 5 : $0.495 > 0.49 \implies$ no improvement

best loss = 0.49, checkpoint = 4, counter = 1

epoch 6 : $0.5 > 0.49 \implies$ no improvement

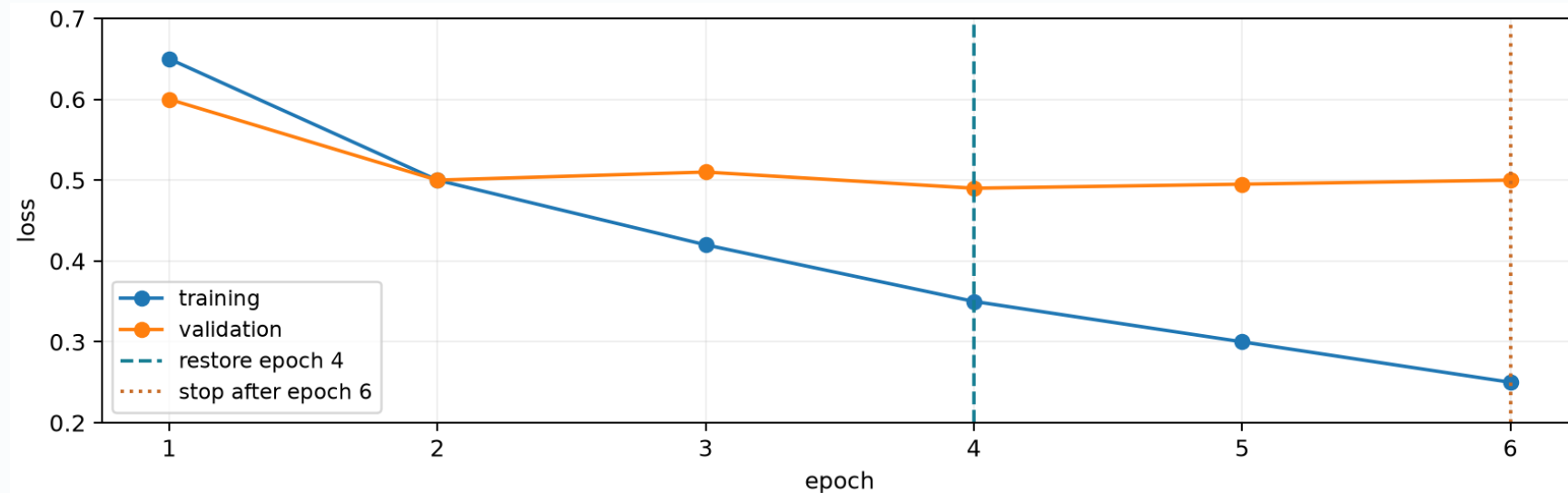
best loss = 0.49, checkpoint = 4, counter = 2

The counter reaches 2 after epoch 6: stop now. The best checkpoint remains epoch 4.

WORKED EXAMPLE

See stopping and restoration as different events

The best validation checkpoint is earlier than the stopping epoch.



Stop after epoch 6, then load epoch 4 parameters. The smaller training loss at epoch 6 is not the checkpoint criterion. This is a selection heuristic, not proof of best possible generalization.

WORKED EXAMPLE

Restore the model, not just a recorded score

The six-epoch run chooses checkpoint 4 for final evaluation.

decision	result
stopping epoch	6
restored checkpoint	4
validation loss of restored model	.49
training loss at that checkpoint	.35

For inference, restore the saved model parameters. If training is deliberately resumed, a faithful training checkpoint also needs optimizer state and counters.

Use validation data for architecture, learning rate and checkpoint choices. Keep the test set out of these choices, then evaluate the selected procedure on it.

WORKED EXAMPLE

Count hyperparameter settings and seeded runs

A configuration chooses one value from each setting list.

$3 \text{ rates} \times 2 \text{ regularization values} \times 2 \text{ batch sizes} = 12$

$12 \text{ configurations} \times 3 \text{ seeds} = 36 \text{ runs}$

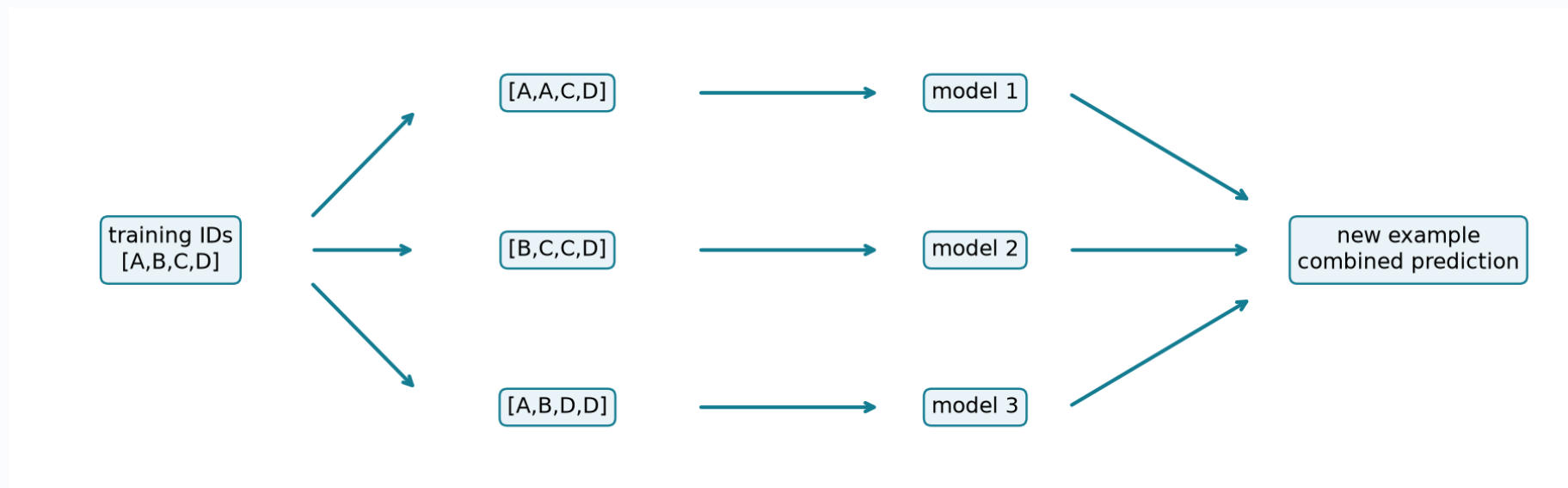
A seed controls the pseudorandom realization, such as initialization or shuffling. Three seeds repeat a configuration; they do not create three new hyperparameter settings.

Choose a validation-based comparison procedure in advance, such as mean validation score across seeds. Do not select settings by repeatedly looking at final test scores.

WORKED EXAMPLE

Bagging: sample training data for several models

A conceptual companion to lecture pages 145-149.



The illustrated bootstrap samples draw training IDs with replacement, so duplicates are allowed. Train separate models, then combine their predictions on a new example. Held-out test examples are not bootstrap training inputs.

WORKED EXAMPLE

Declare how the ensemble combines predictions

Two common aggregation choices are different procedures.

hard predictions = $(1, 0, 1) \Rightarrow \text{majority} = 1$

probabilities = $(0.8, 0.4, 0.7)$

mean probability = $(0.8 + 0.4 + 0.7)/3 = 1.9/3 \approx 0.633333$

threshold 0.5 $\Rightarrow$ mean probability prediction = 1

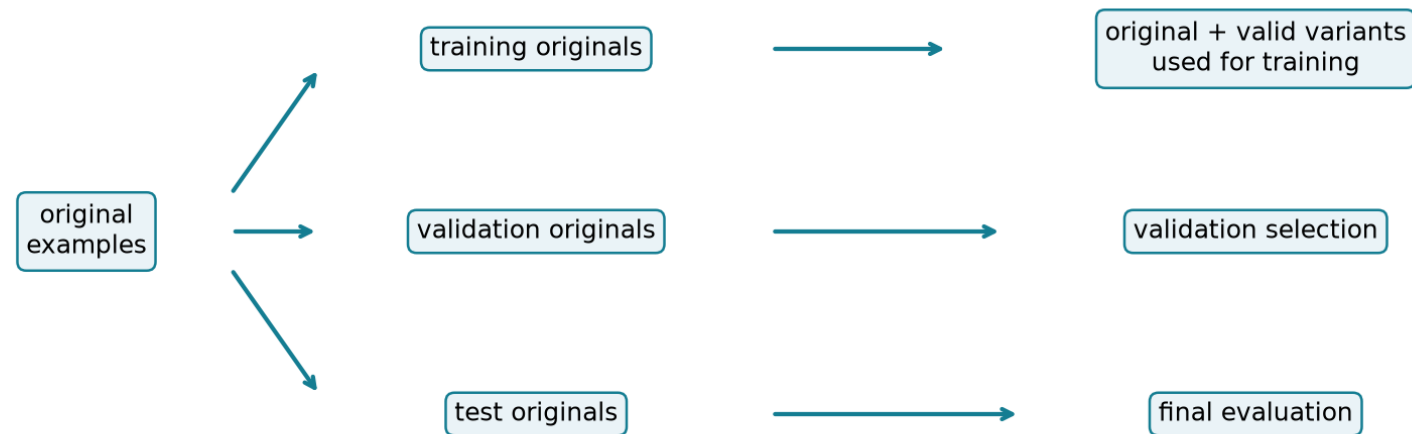
The two choices agree for these numbers but need not always agree. These are supplied model outputs, not a derivation of how their training produced them.

Bagging can reduce variability when models make usefully different errors; it does not guarantee an improvement on every dataset.

WORKED EXAMPLE

Augmentation: split originals before creating variants

A conceptual companion to lecture page 162.



Keep variants of the same original within its assigned partition. Transform training examples only in ways that preserve the target meaning; a digit rotation can change its label.

WORKED EXAMPLE

Count items, not independent originals

Suppose 100 training originals each produce two additional variants.

100 originals + 2(100) variants = 300 training items

These are 300 presented items, not 300 independently collected examples. Sibling variants share information; the independent-sample variance formulas cannot be applied merely by counting files.

Split original examples into training, validation and test groups before producing related variants. Otherwise an original can be in training while a near-copy appears in evaluation.

Check whether a chosen transformation preserves the label for this task. A valid transformation for one image class may be invalid for another.

INDEPENDENT PRACTICE

Your turn: equality and an unobserved future value

Use the same strict-improvement rule with patience 2.

listed validation values = (0.4, 0.35, 0.35, 0.36, 0.30)

Trace the best loss, checkpoint and consecutive-failure counter. When does the run stop, which checkpoint is restored, and is the fifth value actually observed under this rule?

Count runs for a 4-by-3 configuration grid with two seeds per configuration. For bagging votes [0,1,0], state the majority.

If 40 training originals each have three additional label-preserving variants, how many items are presented? When must the split occur?

WORKED ANSWER

Answer: equality does not reset patience

The observed run ends after the fourth epoch.

epoch	validation	best / checkpoint	counter
1	.40	.40 / epoch 1	0
2	.35	.35 / epoch 2	0
3	.35	.35 / epoch 2	1
4	.36	.35 / epoch 2	2: stop

$0.35 = 0.35 \implies$ no strict improvement

stop = 4, restore = 2

The listed fifth value .30 is hypothetical for this stopped run: it would not have been observed. Using it to justify continuing changes the stated procedure.

WORKED ANSWER

Answer: configurations, votes and variants

Keep configuration count, run count and independent-data count distinct.

$4 \times 3 = 12$ configurations; $12 \times 2 = 24$ runs

votes $(0, 1, 0) \Rightarrow \text{majority} = 0$

$40 + 3(40) = 160$ training items

Split the original dataset first; the resulting training partition contains these 40 originals. Then add three variants per training original. The 160 items are not 160 independent originals.

Common errors: restoring the last checkpoint instead of the best, treating equality as improvement, ignoring a counter reset, looking ahead past stopping, tuning on test data, or leaking augmented siblings across partitions.